CS6008 Assignment-02

23b0936, 23b1030

October 2026

1 Phase 1

1.1 Resolution, Fetch and Isolation Check

The client resolved `example.com` through the resolver (172.16.2.10), which returned `172.16.1.20`, and then fetched the page from that address.

```
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase1> docker exec client dig +short example.com
172.16.1.20
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase1> docker exec client dig +short www.example.com
172.16.1.20
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase1> docker exec client curl -s http://example.com
<!DOCTYPE html>
<html>
<head><title>example.com (REAL)</title></head>
<body style="font-family: sans-serif; background:#eef7ee;">
  <h1>This is the REAL example.com</h1>
  <p>Served by webserver (172.16.1.20) - CS6008 Phase 1 baseline.</p>
</body>
</html>
```

Figure 1: Resolution via the resolver and page fetch.

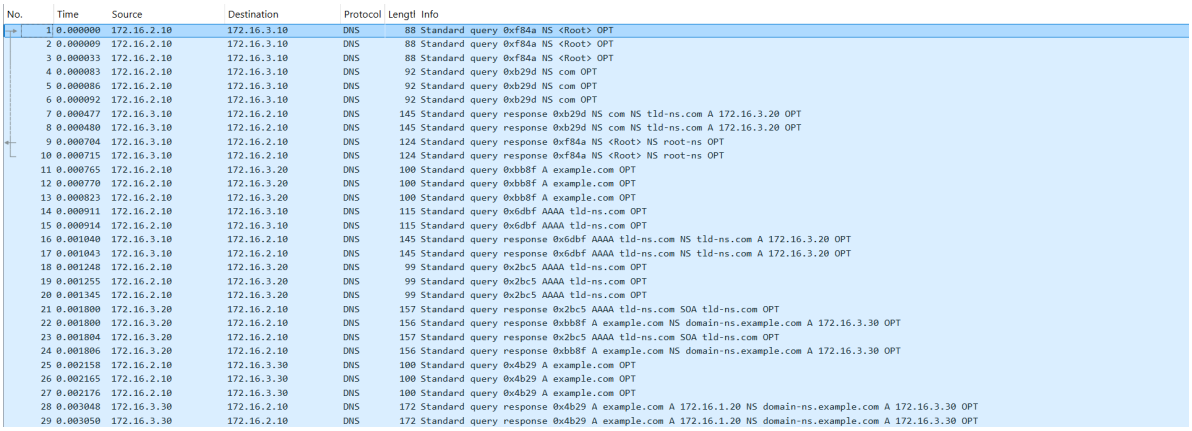
A direct query from the client to root-ns (172.16.3.10) fails: the client has no route to the authoritative subnet 172.16.3.0/24, so all resolution must go through the resolver.

```
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase1> docker exec client dig '@172.16.3.10' example.co
m +noall +comments +answer +stats
;; communications error to 172.16.3.10#53: timed out
;; communications error to 172.16.3.10#53: timed out
;; communications error to 172.16.3.10#53: timed out

; <<>> DiG 9.20.11-0ubuntu0.2-Ubuntu <<>> @172.16.3.10 example.com +noall +comments +answer +stats
; (1 server found)
;; global options: +cmd
;; no servers could be reached
```

Figure 2: Failed direct query to root-ns (isolation check).

1.2 Root → TLD → Domain Referral



No.	Time	Source	Destination	Protocol	Lengtl	Info
1	0.000000	172.16.2.10	172.16.3.10	DNS	88	Standard query 0xf84a NS <Root> OPT
2	0.000009	172.16.2.10	172.16.3.10	DNS	88	Standard query 0xf84a NS <Root> OPT
3	0.000033	172.16.2.10	172.16.3.10	DNS	88	Standard query 0xf84a NS <Root> OPT
4	0.000083	172.16.2.10	172.16.3.10	DNS	92	Standard query 0xb29d NS com OPT
5	0.000086	172.16.2.10	172.16.3.10	DNS	92	Standard query 0xb29d NS com OPT
6	0.000092	172.16.2.10	172.16.3.10	DNS	92	Standard query 0xb29d NS com OPT
7	0.000477	172.16.3.10	172.16.2.10	DNS	145	Standard query response 0xb29d NS com NS tld-ns.com A 172.16.3.20 OPT
8	0.000480	172.16.3.10	172.16.2.10	DNS	145	Standard query response 0xb29d NS com NS tld-ns.com A 172.16.3.20 OPT
9	0.000704	172.16.3.10	172.16.2.10	DNS	124	Standard query response 0xf84a NS <Root> NS root-ns OPT
10	0.000715	172.16.3.10	172.16.2.10	DNS	124	Standard query response 0xf84a NS <Root> NS root-ns OPT
11	0.000765	172.16.2.10	172.16.3.20	DNS	100	Standard query 0xbb8f A example.com OPT
12	0.000770	172.16.2.10	172.16.3.20	DNS	100	Standard query 0xbb8f A example.com OPT
13	0.000823	172.16.2.10	172.16.3.20	DNS	100	Standard query 0xbb8f A example.com OPT
14	0.000911	172.16.2.10	172.16.3.10	DNS	115	Standard query 0x6dbf AAAA tld-ns.com OPT
15	0.000914	172.16.2.10	172.16.3.10	DNS	115	Standard query 0x6dbf AAAA tld-ns.com OPT
16	0.001040	172.16.3.10	172.16.2.10	DNS	145	Standard query response 0x6dbf AAAA tld-ns.com NS tld-ns.com A 172.16.3.20 OPT
17	0.001043	172.16.3.10	172.16.2.10	DNS	145	Standard query response 0x6dbf AAAA tld-ns.com NS tld-ns.com A 172.16.3.20 OPT
18	0.001248	172.16.2.10	172.16.3.20	DNS	99	Standard query 0x2bc5 AAAA tld-ns.com OPT
19	0.001255	172.16.2.10	172.16.3.20	DNS	99	Standard query 0x2bc5 AAAA tld-ns.com OPT
20	0.001345	172.16.2.10	172.16.3.20	DNS	99	Standard query 0x2bc5 AAAA tld-ns.com OPT
21	0.001800	172.16.3.20	172.16.2.10	DNS	157	Standard query response 0x2bc5 AAAA tld-ns.com SOA tld-ns.com OPT
22	0.001800	172.16.3.20	172.16.2.10	DNS	156	Standard query response 0xbb8f A example.com NS domain-ns.example.com A 172.16.3.30 OPT
23	0.001804	172.16.3.20	172.16.2.10	DNS	157	Standard query response 0x2bc5 AAAA tld-ns.com SOA tld-ns.com OPT
24	0.001806	172.16.3.20	172.16.2.10	DNS	156	Standard query response 0xbb8f A example.com NS domain-ns.example.com A 172.16.3.30 OPT
25	0.002158	172.16.2.10	172.16.3.30	DNS	100	Standard query 0x4b29 A example.com OPT
26	0.002165	172.16.2.10	172.16.3.30	DNS	100	Standard query 0x4b29 A example.com OPT
27	0.002176	172.16.2.10	172.16.3.30	DNS	100	Standard query 0x4b29 A example.com OPT
28	0.003048	172.16.3.30	172.16.2.10	DNS	172	Standard query response 0x4b29 A example.com A 172.16.1.20 NS domain-ns.example.com A 172.16.3.30 OPT
29	0.003050	172.16.3.30	172.16.2.10	DNS	172	Standard query response 0x4b29 A example.com A 172.16.1.20 NS domain-ns.example.com A 172.16.3.30 OPT

Figure 3: Referral chain in the capture

The resolver performs iterative resolution (every upstream query carries `RD=0`):

- **Root** (172.16.3.10): query `com. NS` (ID `0xb29d`). Reply is a referral, not an answer: Authority `com. NS tld-ns.com`, Additional (glue) `tld-ns.com A 172.16.3.20`, `AA=0`.
- **TLD** (172.16.3.20): query `example.com A` (ID `0xbb8f`). Reply is a referral: Authority `example.com NS domain-ns.example.com`, glue `domain-ns.example.com A 172.16.3.30`, `AA=0`.
- **Domain** (172.16.3.30): query `example.com A` (ID `0x4b29`). Authoritative answer `example.com A 172.16.1.20`, TTL 86400, `AA=1`.

Every message is cleartext UDP/53 with no DNSSEC. On the wire an observer sees the full query name and type, the 16-bit transaction ID, the resolver's source port, the header flags, the NS and glue records, and the final answer with its TTL.

1.3 Path to root-ns

```
$ ip route
172.16.3.0/24 via <172.16.2.3> dev eth0      # static route
```

The resolver (172.16.2.10) sits on 172.16.2.0/24; the name servers are on 172.16.3.0/24. The static route sends that subnet via the router, which forwards between the two subnets. The capture confirms the hop: outgoing queries leave with `TTL=64` and arrive on the server side with `TTL=63`, and the destination MAC on frame 1 is the router's, not root-ns's.

1.4 On-Path Threat

A node on this path (the router, or any host observing the link) sees the query name, the transaction ID and the source port. It can therefore forge a matching response, or rewrite a root/TLD referral to point `example.com` at a server it controls. The forged record is cached and served to every client of the resolver, and the resolver cannot detect the tampering because there is no DNSSEC. It can equally drop the queries to deny resolution. Phase 2 exploits this.

2 Phase 2 — On-Path Response Forgery

2.1 What the Resolver Cached and What the Fetch Returned

After flushing the cache and resolving `example.com`, the resolver cached the injected record `example.com A 172.16.1.30`, and `curl` returned the rogue server's page. In the same cache, `www.example.com` stayed `172.16.1.20` and still fetched the real page.

```
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase2> docker exec resolver rndc flush
WARNING: key file (/etc/bind/rndc.key) exists, but using default configuration file (/etc/bind/rndc.conf)
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase2> docker exec client dig +short example.com
172.16.1.30
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase2> docker exec client dig +short www.example.com
172.16.1.20
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase2> docker exec client curl -s http://example.com
<!DOCTYPE html>
<html>
<head><title>example.com (ROGUE)</title></head>
<body style="font-family: sans-serif; background:#eef7ee;">
  <h1>This is the ROGUE example.com</h1>
  <p>Served by webserver (172.16.1.20) - CS6008 Phase 1 baseline.</p>
</body>
</html>
```

Figure 4: Poisoned `example.com`→172.16.1.30 (rogue page); `www.example.com`→172.16.1.20 (unaffected).

No.	Time	Source	Destination	Protocol	Lengtl	Info
1	0.000000	172.16.3.10	172.16.2.10	DNS	118	Standard query response 0x2e7b NS <Root> NS root-ns OPT
2	0.000041	172.16.3.10	172.16.2.10	DNS	139	Standard query response 0xee64 NS com NS tld-ns.com A 172.16.3.20 OPT
3	0.032272	172.16.3.10	172.16.2.10	DNS	118	Standard query response 0x2e7b NS <Root> NS root-ns OPT
4	0.032435	172.16.3.10	172.16.2.10	DNS	139	Standard query response 0xee64 NS com NS tld-ns.com A 172.16.3.20 OPT
5	0.033539	172.16.3.10	172.16.2.10	DNS	139	Standard query response 0x3067 AAAA tld-ns.com NS tld-ns.com A 172.16.3.20 OPT
6	0.033596	172.16.3.10	172.16.2.10	DNS	118	Standard query response 0x801e NS <Root> NS root-ns OPT
7	0.035060	172.16.3.20	172.16.2.10	DNS	150	Standard query response 0xcbd4 A example.com NS domain-ns.example.com A 172.16.3.30 OPT
8	0.136226	172.16.3.10	172.16.2.10	DNS	139	Standard query response 0x3067 AAAA tld-ns.com NS tld-ns.com A 172.16.3.20 OPT
9	0.136386	172.16.3.10	172.16.2.10	DNS	118	Standard query response 0x801e NS <Root> NS root-ns OPT
10	0.136402	172.16.3.20	172.16.2.10	DNS	150	Standard query response 0xcbd4 A example.com NS domain-ns.example.com A 172.16.3.30 OPT
11	0.138319	172.16.3.20	172.16.2.10	DNS	151	Standard query response 0xf9c1 AAAA tld-ns.com SOA tld-ns.com OPT
12	0.138751	172.16.3.30	172.16.2.10	DNS	166	Standard query response 0xf72e A example.com A 172.16.1.20 NS domain-ns.example.com A 172.16.3.30 OPT
13	0.240041	172.16.3.20	172.16.2.10	DNS	151	Standard query response 0xf9c1 AAAA tld-ns.com SOA tld-ns.com OPT
14	0.240259	172.16.3.30	172.16.2.10	DNS	166	Standard query response 0xf72e A example.com A 172.16.1.30 NS domain-ns.example.com A 172.16.3.30 OPT
15	7.639388	172.16.3.30	172.16.2.10	DNS	162	Standard query response 0x112a AAAA example.com SOA domain-ns.example.com OPT
16	7.728164	172.16.3.30	172.16.2.10	DNS	162	Standard query response 0x112a AAAA example.com SOA domain-ns.example.com OPT

Figure 5: Annotated capture: genuine answer intercepted and the A record injected.

2.2 Why the Attack Succeeds

Plain DNS over UDP has no authentication or integrity (no DNSSEC): the resolver accepts the first reply that matches the query's transaction ID, source port and question name. An off-path attacker must *guess* the 16-bit ID and the source port and win a race against the real server (the Kaminsky attack). The on-path position removes that guesswork: the attacker *sees* the genuine reply, so it already knows the ID, port and question, and simply edits the real packet in flight. The forged answer is therefore guaranteed to match and to be accepted.

2.3 How Non-Target Names Stay Untouched

The relay parses each message and inspects the question. Only a response whose QNAME is exactly `example.com` type A is rewritten; every other packet is forwarded byte-for-byte. `www.example.com` is a different QNAME, so it is never matched and passes through unchanged — which is why it still resolves to the genuine `172.16.1.20`.

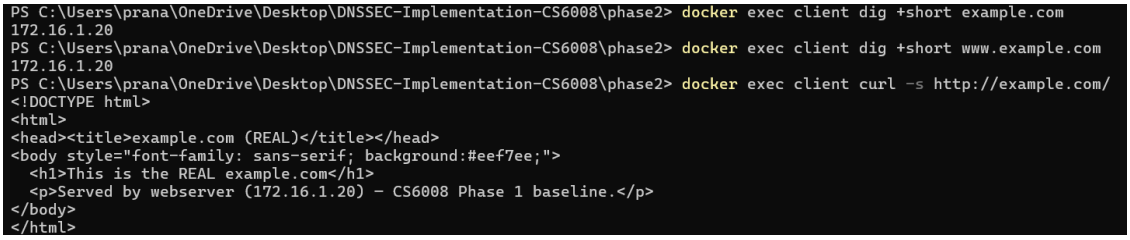


Figure 6: Unaffected non-target lookup `www.example.com`→172.16.1.20.

3 Phase 3 — DNSSEC Chain of Trust

3.1 Healthy Signed Hierarchy (Verify)

With no interceptor in the path, `dig +dnssec example.com` returns `NOERROR` with the `ad` (authenticated-data) flag set and a valid `RRSIG` over `example.com A 172.16.1.20`. Validating the full chain with `delv` against the root KSK as trust anchor reports `fully validated`, confirming an unbroken chain from the root key down to `example.com`.

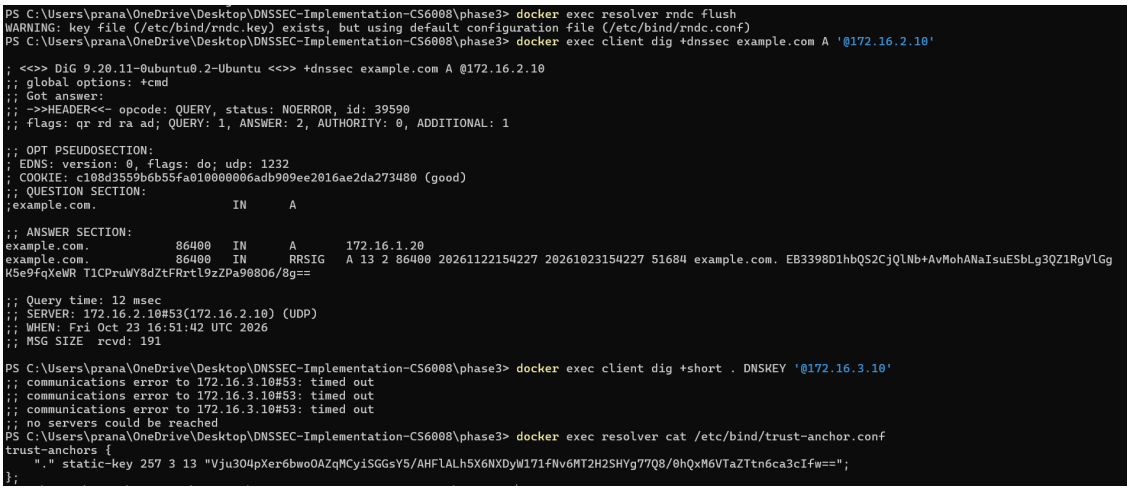


Figure 7: `dig +dnssec`: `ad` flag and valid RRSIG.

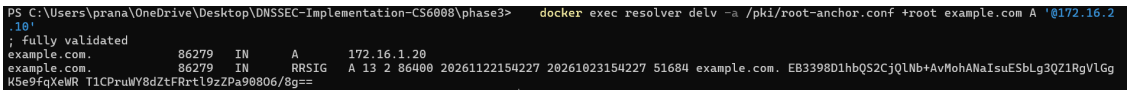


Figure 8: `delv` validates the full chain against the root trust anchor.

3.2 What the Resolver Does With the Forged Response

With forging enabled and the cache flushed, `dig +dnssec example.com` now returns `SERVFAIL` with *no* `ad` flag, and `curl` fails with `Could not resolve host`. The attacker still intercepts and injects `A 172.16.1.30` exactly as in Phase 2, but the resolver rejects it.

```
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase3> docker exec resolver rndc flush
WARNING: key file (/etc/bind/rndc.key) exists, but using default configuration file (/etc/bind/rndc.conf)
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase3>    docker exec client dig +dnssec example.com A '@172.16.2.10'

; <<>> DiG 9.20.11-0ubuntu0.2-Ubuntu <<>> +dnssec example.com A @172.16.2.10
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL, id: 44836
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: 3e995fd42777ea4c010000006adb918223f2ca883e151609 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; Query time: 500 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Fri Oct 23 16:55:30 UTC 2026
;; MSG SIZE  rcvd: 68

PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase3>    docker exec client curl -sv http://example.com/
* Could not resolve host: example.com
* shutting down connection #0
```

Figure 9: Attack blocked: SERVFAIL and failed fetch.

No.	Time	Source	Destination	Protocol	Lengtl	Info
1	0.000000	172.16.3.10	172.16.2.10	DNS	212	Standard query response 0x2759 NS <Root> NS root-ns RRSIG OPT
2	0.000524	172.16.3.10	172.16.2.10	DNS	282	Standard query response 0x88b4 NS com NS tld-ns.com DS RRSIG A 172.16.3.20 OPT
3	0.103661	172.16.3.10	172.16.2.10	DNS	212	Standard query response 0x2759 NS <Root> NS root-ns RRSIG OPT
4	0.103855	172.16.3.10	172.16.2.10	DNS	282	Standard query response 0x88b4 NS com NS tld-ns.com DS RRSIG A 172.16.3.20 OPT
5	0.105539	172.16.3.10	172.16.2.10	DNS	444	Standard query response 0x25e0 DNSKEY <Root> DNSKEY DNSKEY RRSIG RRSIG OPT
6	0.105614	172.16.3.10	172.16.2.10	DNS	282	Standard query response 0xe9e4 AAAA tld-ns.com NS tld-ns.com DS RRSIG A 172.16.3.20 OPT
7	0.108174	172.16.3.20	172.16.2.10	DNS	297	Standard query response 0x3c7c A example.com NS domain-ns.example.com DS RRSIG A 172.16.3.30 OPT
8	0.207532	172.16.3.10	172.16.2.10	DNS	444	Standard query response 0x25e0 DNSKEY <Root> DNSKEY DNSKEY RRSIG RRSIG OPT
9	0.207704	172.16.3.10	172.16.2.10	DNS	282	Standard query response 0xe9e4 AAAA tld-ns.com NS tld-ns.com DS RRSIG A 172.16.3.20 OPT
10	0.207721	172.16.3.20	172.16.2.10	DNS	297	Standard query response 0x3c7c A example.com NS domain-ns.example.com DS RRSIG A 172.16.3.30 OPT
11	0.208851	172.16.3.20	172.16.2.10	DNS	374	Standard query response 0xcd5c AAAA tld-ns.com SOA tld-ns.com RRSIG NSEC com RRSIG OPT
12	0.209349	172.16.3.30	172.16.2.10	DNS	487	Standard query response 0xc87e A example.com A 172.16.1.20 RRSIG NS domain-ns.example.com RRSIG A 172.16.3.30 RRSIG OPT
13	0.311498	172.16.3.20	172.16.2.10	DNS	374	Standard query response 0xcd5c AAAA tld-ns.com SOA tld-ns.com RRSIG NSEC com RRSIG OPT
14	0.311728	172.16.3.30	172.16.2.10	DNS	487	Standard query response 0xc87e A example.com A 172.16.1.30 RRSIG NS domain-ns.example.com RRSIG A 172.16.3.30 RRSIG OPT
15	0.314711	172.16.3.30	172.16.2.10	DNS	484	Standard query response 0xed80 DNSKEY example.com DNSKEY DNSKEY RRSIG RRSIG OPT
16	0.415570	172.16.3.30	172.16.2.10	DNS	484	Standard query response 0xed80 DNSKEY example.com DNSKEY DNSKEY RRSIG RRSIG OPT
17	0.419496	172.16.3.20	172.16.2.10	DNS	460	Standard query response 0xf1ad DNSKEY com DNSKEY DNSKEY RRSIG RRSIG OPT
18	0.519701	172.16.3.20	172.16.2.10	DNS	460	Standard query response 0xf1ad DNSKEY com DNSKEY DNSKEY RRSIG RRSIG OPT
19	0.523797	172.16.3.10	172.16.2.10	DNS	300	Standard query response 0x0646 AAAA domain-ns.example.com NS tld-ns.com DS RRSIG A 172.16.3.20 OPT
20	0.524020	172.16.3.10	172.16.2.10	DNS	212	Standard query response 0xc5ea NS <Root> NS root-ns RRSIG OPT
21	0.623414	172.16.3.10	172.16.2.10	DNS	300	Standard query response 0x0646 AAAA domain-ns.example.com NS tld-ns.com DS RRSIG A 172.16.3.20 OPT
22	0.623596	172.16.3.10	172.16.2.10	DNS	212	Standard query response 0xc5ea NS <Root> NS root-ns RRSIG OPT
23	0.624976	172.16.3.20	172.16.2.10	DNS	297	Standard query response 0xd0cd AAAA domain-ns.example.com NS domain-ns.example.com DS RRSIG A 172.16.3.30 OPT
24	0.727467	172.16.3.20	172.16.2.10	DNS	297	Standard query response 0xd0cd AAAA domain-ns.example.com NS domain-ns.example.com DS RRSIG A 172.16.3.30 OPT
25	0.728493	172.16.3.30	172.16.2.10	DNS	413	Standard query response 0x2f3a AAAA domain-ns.example.com SOA domain-ns.example.com RRSIG NSEC www.example.com RRSIG OPT
26	0.831654	172.16.3.30	172.16.2.10	DNS	413	Standard query response 0x2f3a AAAA domain-ns.example.com SOA domain-ns.example.com RRSIG NSEC www.example.com RRSIG OPT
27	5.681816	172.16.3.30	172.16.2.10	DNS	487	Standard query response 0x2788 A example.com A 172.16.1.20 RRSIG NS domain-ns.example.com RRSIG A 172.16.3.30 RRSIG OPT
28	5.681940	172.16.3.30	172.16.2.10	DNS	420	Standard query response 0xab56 AAAA example.com SOA domain-ns.example.com RRSIG NSEC domain-ns.example.com RRSIG OPT
29	5.719566	172.16.3.30	172.16.2.10	DNS	487	Standard query response 0x2788 A example.com A 172.16.1.30 RRSIG NS domain-ns.example.com RRSIG A 172.16.3.30 RRSIG OPT
30	5.719693	172.16.3.30	172.16.2.10	DNS	420	Standard query response 0xab56 AAAA example.com SOA domain-ns.example.com RRSIG NSEC domain-ns.example.com RRSIG OPT

Figure 10: Capture: forged `A 172.16.1.30` injected, but carrying no valid RRSIG.

The difference from Phase 2 is validation. There, the resolver accepted the first answer that matched the transaction ID, source port and question. Here the resolver additionally verifies the `RRSIG` covering the A record against the `example.com` ZSK, that key against the `example.com` DS in `com`, and so on up to the root KSK it trusts. The forged record carries no valid signature under that chain, so it is discarded as *bogus* and the resolver returns SERVFAIL rather than the injected address.

3.3 On the Wire vs. Phase 1

The exchange is still plaintext UDP/53. The on-path attacker can read exactly what it could in Phase 1: the full query name and type, the transaction ID, the source port, the flags, and the answer records (now with their RRSIGs). DNSSEC adds *origin authentication and integrity* — it proves the data was signed by the zone's key and not altered — so tampering is detected and rejected. It does *not* add confidentiality: the attacker still sees every name looked up, and DNSSEC does not stop it from dropping packets to deny resolution.

3.4 Signing Algorithm and Key Handling

The zones are signed with **ECDSAP256SHA256** (algorithm 13). Its trade-offs: small keys and signatures and fast validation compared with RSA, keeping responses compact and avoiding fragmentation, at the cost of requiring reasonably modern resolver support (universal today). Only the *public* KSK and the DS records

are committed to the repository; the **private** zone-signing keys (`K.+013+*.private`) are never committed, since anyone holding them could forge valid signatures for the zone and defeat the chain of trust.

4 Phase 4 — Signature Stripping & Downgrade

4.1 Authenticated Baseline

With the Phase 3 signed hierarchy and validating resolver unchanged and no stripping active, the client resolves `example.com` as an authenticated answer: `dig +dnssec` returns `172.16.1.20` with the `ad` flag and a valid `RRSIG`, and `delv` reports `fully validated`.

```
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase4> docker exec resolver rndc flush
WARNING: key file (/etc/bind/rndc.key) exists, but using default configuration file (/etc/bind/rndc.con
f)
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase4> docker exec client dig +dnssec
example.com A '@172.16.2.10'

; <<>> DiG 9.20.11-0ubuntu0.2-Ubuntu <<>> +dnssec example.com A @172.16.2.10
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 22180
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: 46cc457e2a0ff366010000006adb94c663a38f95ed44a254 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            86400   IN      A       172.16.1.20
example.com.            86400   IN      RRSIG   A 13 2 86400 20261122160734 20261023160734 1238 example
.com. 0UOpyY+v4ZPSMtrzkL/lumtjg2PvU+u4utwbUpsnPeDmCZql54tS10JL AE9xE7s3la12VzPkJlyxEgJm+uVFiw==

;; Query time: 12 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Fri Oct 23 17:09:26 UTC 2026
;; MSG SIZE  rcvd: 191
```

Figure 11: Authenticated baseline before stripping.

4.2 Run A — Strip Records from a Securely Delegated Child

The interceptor, in strip mode, removes the `DNSKEY` and `RRSIG` records for `example.com` (and the `com` referral's DS) and injects a forged `A 172.16.1.30`. The parent `com` still publishes the child's DS. The resolver returns **SERVFAIL** with no `ad` flag; `delv` fails validation and `curl` cannot resolve the host.

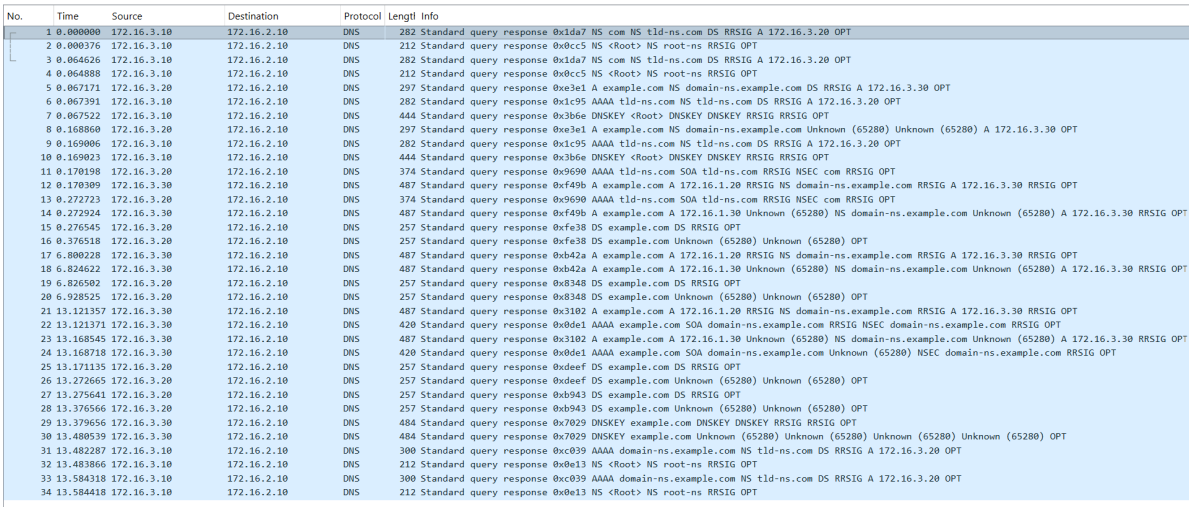
```
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase4> docker exec resolver rndc flush

WARNING: key file (/etc/bind/rndc.key) exists, but using default configuration file (/etc/bind/rndc.conf)
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase4> docker exec client dig '@172.16.2.10' example.com A +dnssec +noall +comments +answer
 +stats
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: SERVFAIL, id: 40913
;; flags: qr rd ra; QUERY: 1, ANSWER: 0, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: fd5e52bcc7ebb00f010000006adb9572f6130f5149df273b (good)
;; Query time: 380 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Fri Oct 23 17:12:18 UTC 2026
;; MSG SIZE  rcvd: 68

PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase4> docker exec client delv -a /pki/root-anchor.conf '@172.16.2.10' example.com A +rtrac
e
;; fetch: example.com/A
;; resolution failed: failure
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase4> docker exec client curl -i --max-time 10 http://example.com/
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
  0     0    0     0    0     0      0      0 --:--:-- --:--:-- --:--:--     0curl: (6) Could not resolve host: example.com
```

Figure 12: Run A result: stripped answer rejected (SERVFAIL, failed fetch).



No.	Time	Source	Destination	Protocol	Lengtl	Info
1	0.000000	172.16.3.10	172.16.2.10	DNS	282	Standard query response 0x1da7 NS com NS tld-ns.com DS RRSIG A 172.16.3.20 OPT
2	0.000376	172.16.3.10	172.16.2.10	DNS	212	Standard query response 0x0cc5 NS <Root> NS root-ns RRSIG OPT
3	0.064626	172.16.3.10	172.16.2.10	DNS	282	Standard query response 0x1da7 NS com NS tld-ns.com DS RRSIG A 172.16.3.20 OPT
4	0.064888	172.16.3.10	172.16.2.10	DNS	212	Standard query response 0x0cc5 NS <Root> NS root-ns RRSIG OPT
5	0.067171	172.16.3.20	172.16.2.10	DNS	297	Standard query response 0xe3e1 A example.com NS domain-ns.example.com DS RRSIG A 172.16.3.30 OPT
6	0.067391	172.16.3.10	172.16.2.10	DNS	282	Standard query response 0x1c95 AAAA tld-ns.com NS tld-ns.com DS RRSIG A 172.16.3.20 OPT
7	0.067522	172.16.3.10	172.16.2.10	DNS	444	Standard query response 0x3b6e DNSKEY <Root> DNSKEY DNSKEY RRSIG RRSIG OPT
8	0.168860	172.16.3.20	172.16.2.10	DNS	297	Standard query response 0xe3e1 A example.com NS domain-ns.example.com Unknown (65280) Unknown (65280) A 172.16.3.30 OPT
9	0.169006	172.16.3.10	172.16.2.10	DNS	282	Standard query response 0x1c95 AAAA tld-ns.com NS tld-ns.com DS RRSIG A 172.16.3.20 OPT
10	0.169023	172.16.3.10	172.16.2.10	DNS	444	Standard query response 0x3b6e DNSKEY <Root> DNSKEY DNSKEY RRSIG RRSIG OPT
11	0.170198	172.16.3.20	172.16.2.10	DNS	374	Standard query response 0x9690 AAAA tld-ns.com SOA tld-ns.com RRSIG NSEC com RRSIG OPT
12	0.170309	172.16.3.30	172.16.2.10	DNS	487	Standard query response 0xf49b A example.com A 172.16.1.20 RRSIG NS domain-ns.example.com RRSIG A 172.16.3.30 RRSIG OPT
13	0.272723	172.16.3.20	172.16.2.10	DNS	374	Standard query response 0x9690 AAAA tld-ns.com SOA tld-ns.com RRSIG NSEC com RRSIG OPT
14	0.272924	172.16.3.30	172.16.2.10	DNS	487	Standard query response 0xf49b A example.com A 172.16.1.30 Unknown (65280) NS domain-ns.example.com Unknown (65280) A 172.16.3.30 RRSIG OPT
15	0.276545	172.16.3.20	172.16.2.10	DNS	257	Standard query response 0xfe38 DS example.com DS RRSIG OPT
16	0.376518	172.16.3.20	172.16.2.10	DNS	257	Standard query response 0xfe38 DS example.com Unknown (65280) Unknown (65280) OPT
17	6.800228	172.16.3.30	172.16.2.10	DNS	487	Standard query response 0xb42a A example.com A 172.16.1.20 RRSIG NS domain-ns.example.com RRSIG A 172.16.3.30 RRSIG OPT
18	6.824622	172.16.3.30	172.16.2.10	DNS	487	Standard query response 0xb42a A example.com A 172.16.1.30 Unknown (65280) NS domain-ns.example.com Unknown (65280) A 172.16.3.30 RRSIG OPT
19	6.826502	172.16.3.20	172.16.2.10	DNS	257	Standard query response 0x8348 DS example.com DS RRSIG OPT
20	6.928525	172.16.3.20	172.16.2.10	DNS	257	Standard query response 0x8348 DS example.com Unknown (65280) Unknown (65280) OPT
21	13.121357	172.16.3.30	172.16.2.10	DNS	487	Standard query response 0x3102 A example.com A 172.16.1.20 RRSIG NS domain-ns.example.com RRSIG A 172.16.3.30 RRSIG OPT
22	13.121371	172.16.3.30	172.16.2.10	DNS	420	Standard query response 0x0de1 AAAA example.com SOA domain-ns.example.com RRSIG NSEC domain-ns.example.com RRSIG OPT
23	13.168545	172.16.3.30	172.16.2.10	DNS	487	Standard query response 0x3102 A example.com A 172.16.1.30 Unknown (65280) NS domain-ns.example.com Unknown (65280) A 172.16.3.30 RRSIG OPT
24	13.168718	172.16.3.30	172.16.2.10	DNS	420	Standard query response 0x0de1 AAAA example.com SOA domain-ns.example.com Unknown (65280) NSEC domain-ns.example.com RRSIG OPT
25	13.171135	172.16.3.20	172.16.2.10	DNS	257	Standard query response 0xdeef DS example.com DS RRSIG OPT
26	13.272665	172.16.3.20	172.16.2.10	DNS	257	Standard query response 0xdeef DS example.com Unknown (65280) Unknown (65280) OPT
27	13.275641	172.16.3.20	172.16.2.10	DNS	257	Standard query response 0xb943 DS example.com DS RRSIG OPT
28	13.376566	172.16.3.20	172.16.2.10	DNS	257	Standard query response 0xb943 DS example.com Unknown (65280) Unknown (65280) OPT
29	13.379656	172.16.3.30	172.16.2.10	DNS	484	Standard query response 0x7029 DNSKEY example.com DNSKEY DNSKEY RRSIG RRSIG OPT
30	13.480539	172.16.3.30	172.16.2.10	DNS	484	Standard query response 0x7029 DNSKEY example.com Unknown (65280) Unknown (65280) Unknown (65280) Unknown (65280) OPT
31	13.482287	172.16.3.10	172.16.2.10	DNS	300	Standard query response 0xc039 AAAA domain-ns.example.com NS tld-ns.com DS RRSIG A 172.16.3.20 OPT
32	13.483866	172.16.3.10	172.16.2.10	DNS	212	Standard query response 0x0e13 NS <Root> NS root-ns RRSIG OPT
33	13.584318	172.16.3.10	172.16.2.10	DNS	300	Standard query response 0xc039 AAAA domain-ns.example.com NS tld-ns.com DS RRSIG A 172.16.3.20 OPT
34	13.584418	172.16.3.10	172.16.2.10	DNS	212	Standard query response 0x0e13 NS <Root> NS root-ns RRSIG OPT

Figure 13: Run A capture: DNSSEC records stripped and forged A injected, yet rejected.

The attacker *cannot make the zone look unsigned*. The resolver's root anchor gives it a secure path through the signed `com` zone, which contains a signed DS for `example.com`: the resolver therefore *knows* the child is signed and requires valid signatures. Simply deleting the RRSIGs, DNSKEY or DS on the wire does not produce the authenticated proof (a signed NSEC/NSEC3 denial) that *no* DS exists, so the missing data is treated as **bogus**, not as permission to accept unsigned data. Stripping alone cannot forge that signed "insecure" proof.

4.3 Run B — Signed Child, Missing Parent DS

The single deployment change is that `com` publishes *no* DS for `example.com` (`INSERT_CHILD_DS=false`); the child is still fully signed, and the same strip-and-forge attack runs. The `DS` lookup returns no DS together with a signed denial from `com`; the A lookup now returns `172.16.1.30` (without the `ad` flag) and `curl` fetches the rogue page.

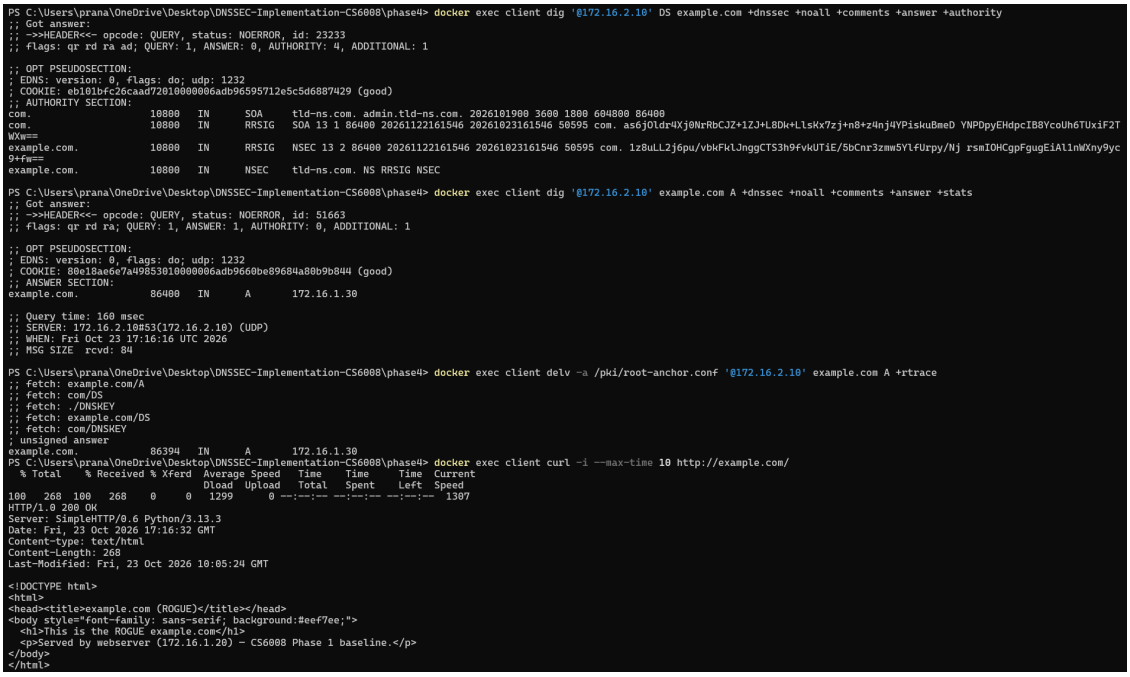


Figure 14: Run B result: no DS, forged `172.16.1.30`, rogue page served.



Figure 15: Run B capture: stripped answer with forged A, accepted by the resolver.

The attack reopens because the parent now provides a *signed proof of absence* of a DS: the resolver validates that proof, legitimately concludes that `example.com` is an **insecure** (unsigned) delegation, and so ignores the child's signatures entirely. With no secure link into the child, the stripped, forged answer is accepted as ordinary unsigned data.

4.4 Where Downgrade Resistance Comes From

The contrast shows that downgrade resistance does *not* come from signatures merely existing in the child zone — those are present in both runs. It comes from the **authenticated chain of trust**: specifically the parent's signed DS record (or a signed proof of its absence). As long as the parent securely asserts "this child is signed, here is its DS," an on-path attacker cannot strip its way to an unsigned state. The exact operator mistake that forfeits this is **signing the child zone but never publishing its DS in the parent**: the child looks secure to its operator, but to a validating resolver it is indistinguishable from an unsigned zone, and its signatures provide no protection.

5 Phase 5 — Zone Walking

5.1 Verify Authenticated Denial (NSEC)

The Phase 3 signed, validating deployment is reused, with `example.com` signed using NSEC denial and five undisclosed internal names added. Querying a name that does not exist returns an authenticated `NXDOMAIN`: the `ad` flag is set and the authority section carries `NSEC` plus `RRSIG` records, which `delv` validates as a fully-validated negative answer.

```
WARNING: key file (/etc/bind/rndc.key) exists, but using default configuration file (/etc/bind/rndc.conf)
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase5> docker compose exec client dig '@172.16.2.10' not-real.example.com A +dnssec +noall
+comments +answer +authority
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 8769
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 0, AUTHORITY: 6, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags: do; udp: 1232
; COOKIE: 5c89b4c63a17b31d010000006adb99dda6ad1cf43165ee59 (good)
;; AUTHORITY SECTION:
mail.example.com.       10800   IN      RRSIG   NSEC 13 3 86400 20261122163043 20261023163043 25080 example.com. JcCiSjRLE7T2iqc+cpbHn/AFWaWC+RyCkIwjVCT1X0w
Y9z7MFKT8F1Tz WCAY5gS+ByS99UjfBk9J8LABJ1Sx1w==
mail.example.com.       10800   IN      NSEC    vpn.example.com. A RRSIG NSEC
example.com.            10800   IN      SOA     domain-ns.example.com. admin.example.com. 2026102300 3600 1800 604800 86400
example.com.            10800   IN      RRSIG   SOA 13 2 86400 20261122163043 20261023163043 25080 example.com. cHy7dzh6xS8wg+lBmotpeASum8hq7AwpBD2e297XRypA
doyGj/hPJkLX o4Ba9/SMdan9tJ//tHuaIw2OmjF0Zw==
example.com.            10800   IN      RRSIG   NSEC 13 2 86400 20261122163043 20261023163043 25080 example.com. 6SoMx5oPAW53Qq4zLZW7jCq5+w87/Nm9tNPwK+K6wD6
JwUepJJ3QBrXL ku9f46i4E+4MhTKT3xgf8+3OA1P1OQ==
example.com.            10800   IN      NSEC    admin.example.com. A NS SOA RRSIG NSEC DNSKEY

PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase5> docker compose exec client delv -a /pki/root-anchor.conf '@172.16.2.10' not-real.exa
mple.com A +rtrace
;; fetch: not-real.example.com/A
;; fetch: example.com/DNSKEY
;; fetch: example.com/DS
;; fetch: com/DNSKEY
;; fetch: com/DS
;; fetch: ./DNSKEY
;; resolution failed: ncache nxdomain
; negative response, fully validated
; not-real.example.com. 10794   IN      \-ANY   ;-$NXDOMAIN
; mail.example.com. RRSIG NSEC ...
; mail.example.com. NSEC vpn.example.com. A RRSIG NSEC
; example.com. SOA domain-ns.example.com. admin.example.com. 2026102300 3600 1800 604800 86400
; example.com. RRSIG SOA ...
; example.com. RRSIG NSEC ...
; example.com. NSEC admin.example.com. A NS SOA RRSIG NSEC DNSKEY
```

Figure 16: Authenticated NSEC denial for a nonexistent name.

5.2 The Walk

An `NSEC` record names both its current owner and the *next* existing owner in canonical order. Starting only from the apex `example.com` and querying a nonexistent type (`TXT`) at each owner, the next-owner field of each reply points to the next name; following the chain until it wraps back to the apex recovers every name.

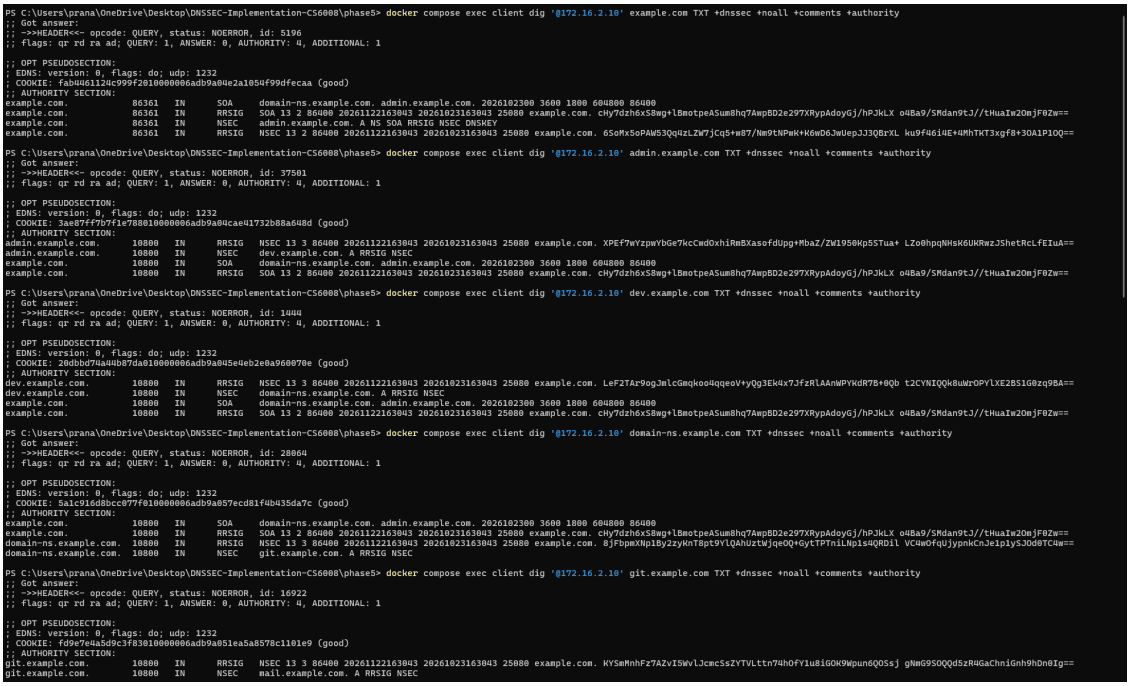


Figure 17: Walking the NSEC chain (part 1).

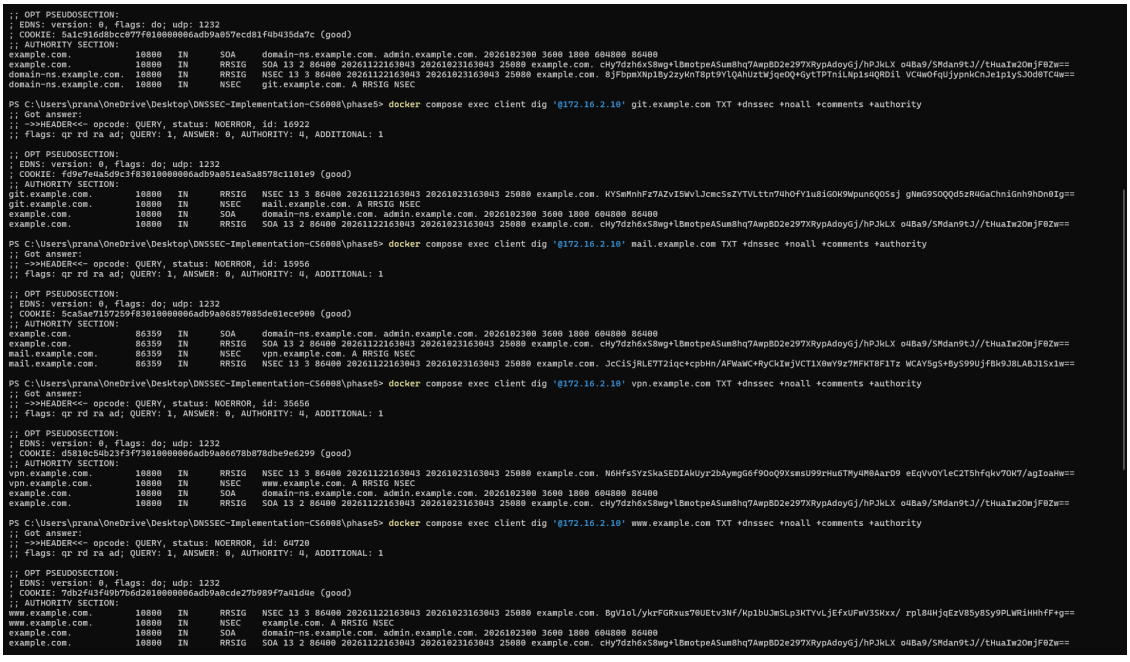


Figure 18: Walking the NSEC chain (part 2), wrapping back to the apex.

The recovered chain, in eight queries (within the `names`+2 limit), is:

`example.com` → `admin` → `dev` → `domain-ns` → `git` → `mail` → `vpn` → `www` → `example.com`.

The five previously undisclosed names are **admin, dev, git, mail, vpn** (alongside the apex, `www` and `domain-ns`). No AXFR or wordlist was used — only authenticated NSEC responses.

5.3 The Fix (NSEC3)

Re-signing `example.com` with NSEC3 (`dnssec-signzone -3 <salt>`, salt `a1b2c3d4`) keeps denial authenticated while closing the leak. A nonexistent name still gets `NXDOMAIN` with the `ad` flag and a validated `NSEC3`/`RRSIG` proof.

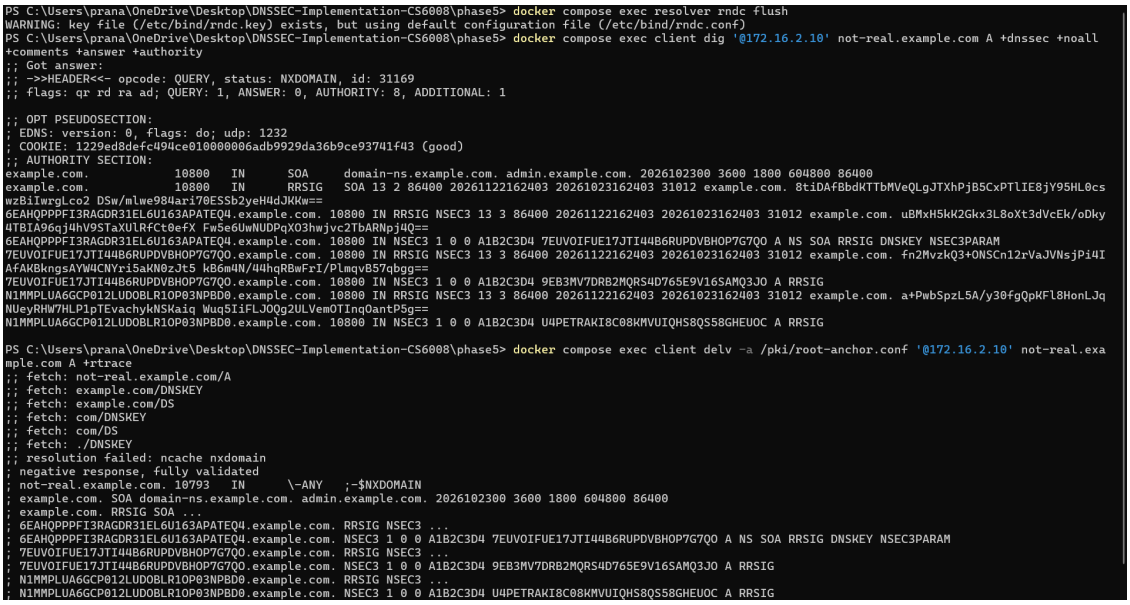


Figure 19: Authenticated denial still works under NSEC3.

But the NSEC3 record now lists *hashed* owner and next-owner values instead of cleartext links, so there is no hostname to follow and the same procedure recovers none of the five labels.

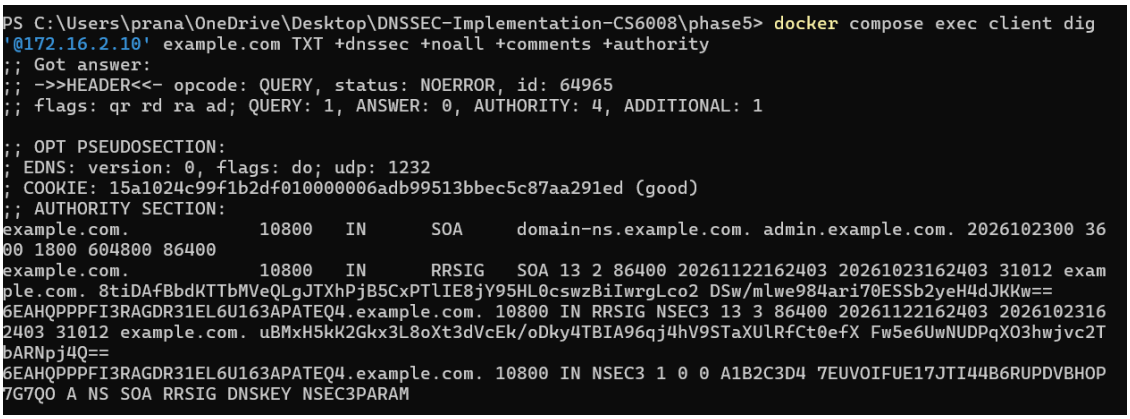


Figure 20: Under NSEC3 the walk yields only hashes — no names recovered.

5.4 Report

Why the walk is possible, and why signed denial leaks. Default NSEC signing made the walk possible. A signed denial must cover *arbitrary* names not known when the zone was signed, so it cannot be about the queried name itself — it must instead prove a gap between two names that *do* exist. NSEC does this with a signed, cleartext link from each owner to the next, and those links chained together are the whole zone. Ordinary DNS answers never publish such a chain: they only ever name records that were explicitly asked for, so there is nothing to enumerate.

What the fix costs, and what remains. NSEC3 replaces the cleartext link with salted hashes of the owner names. The operator cost is extra hashing during signing and validation, larger denial records (and the opt-out/iteration and salt-rotation lifecycle to manage). It is not absolute confidentiality: an enumerator can still collect the NSEC3 hashes and run an *offline dictionary attack*, recovering any predictable or common hostnames by hashing candidates with the published salt and matching them.

6 Phase 6 — Last-Mile Response Forgery

6.1 Setup

The Phase 3 signed, validating deployment is reused with the upstream untouched. The attacker is relocated to the client subnet at `172.16.1.100`, and the edge-router installs a per-client host route (`172.16.1.10/32` and `172.16.1.11/32`) via the attacker, diverting resolver→client replies through it. The attacker relays each reply and, for `example.com`, rewrites only the A rdata to `172.16.1.30` (rogue-web), leaving the genuine RRSIG in place. Two clients are tested: Client A (ordinary stub, `172.16.1.10`) and Client B (locally validating with `delv` against the root KSK, `172.16.1.11`).

6.2 Verify (Transparent Relay)

With forging off, both clients still get the real answer through the diversion: Client A resolves `172.16.1.20` and fetches the real page, and Client B reports `fully validated`.

```
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase6> docker compose up -d --force-recreate edge-router attacker
[+] up 2/2
 ✔Container edge-router Started                                  10.9s
 ✔Container attacker    Started                                  10.9s
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase6> docker exec resolver rndc flush
WARNING: key file (/etc/bind/rndc.key) exists, but using default configuration file (/etc/bind/rndc.conf)
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase6> docker exec client dig '@172.16.2.10' example.com +short            # -> 172.16.1.20
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase6> docker exec client curl -s --max-time 10 http://example.com/       # real page
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase6> docker exec client-b delv -a /pki/root-anchor.conf '@172.16.2.10' example.com
;; resolution failed: failure
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase6>
```

Figure 21: Transparent relay: both clients get the genuine answer.

6.3 Attack — Outcome Per Client

With forging enabled, the two clients diverge:

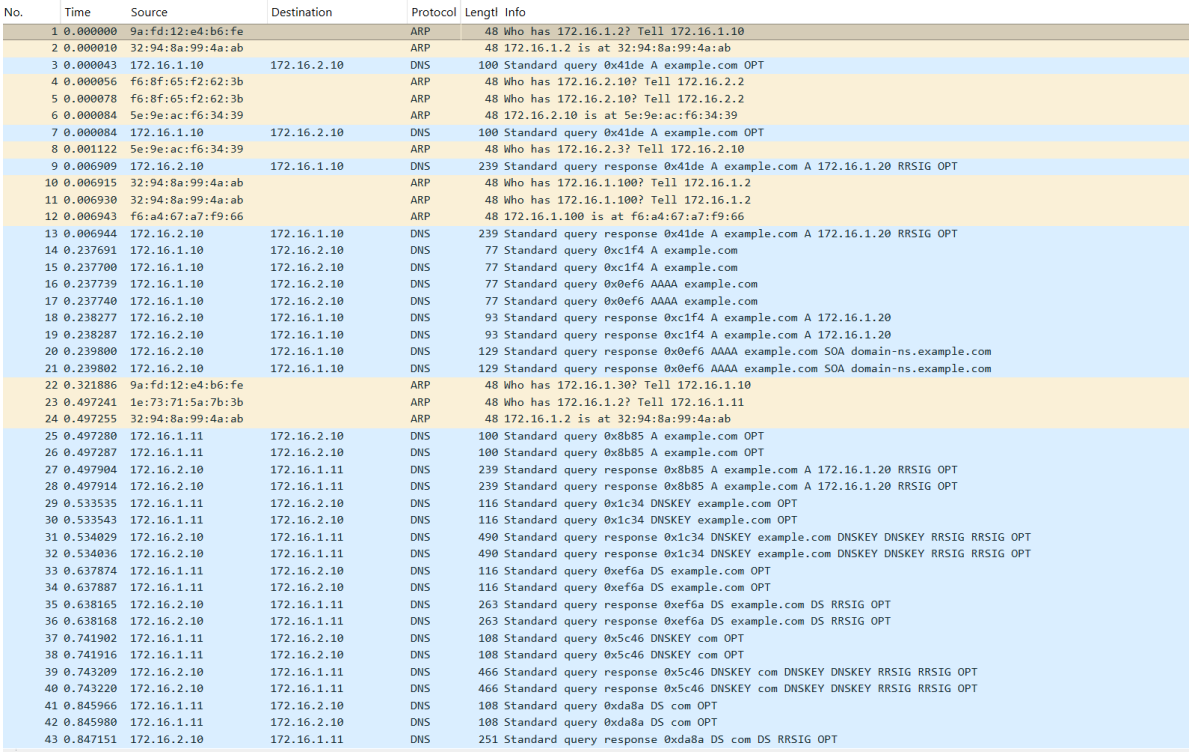
- **Client A (stub)** is fooled: `dig` returns `172.16.1.30` and `curl` lands on the rogue page.
- **Client B (validating)** refuses: local validation fails (`no valid RRSIG` / SERVFAIL) and `delv` does not hand back `172.16.1.30`.

```
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase6> docker exec client curl -s --max-time 10 http://example.com/
<!DOCTYPE html>
<html>
<head><title>example.com (ROGUE)</title></head>
<body style="font-family: sans-serif; background:#eef7ee;">
  <h1>This is the ROGUE example.com</h1>
  <p>Served by rogue-web (172.16.1.30) - CS6008 Phase 6.</p>
</body>
</html>
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase6> docker exec client dig '@172.16.2.10' example.com +short
172.16.1.30
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase6>
```

Figure 22: Client A fooled: forged `172.16.1.30` and rogue page.

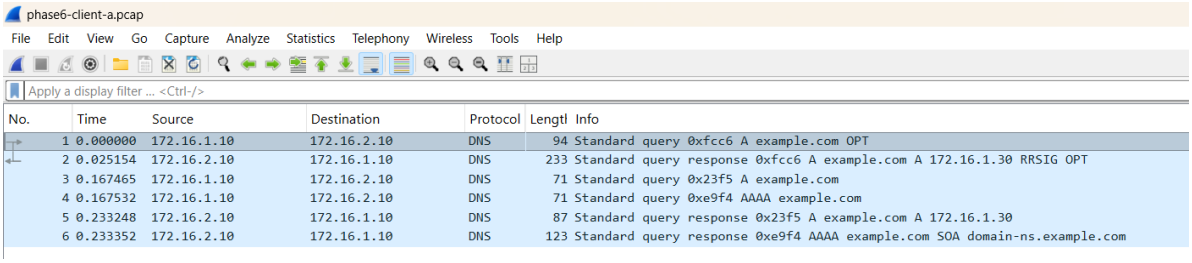
```
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase6> docker exec client-b delv -a /pki/root-anchor.conf '@172.16.2.10' example.com
;; resolution failed: failure
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase6>
```

Figure 23: Client B refuses: local validation fails on the forged answer.



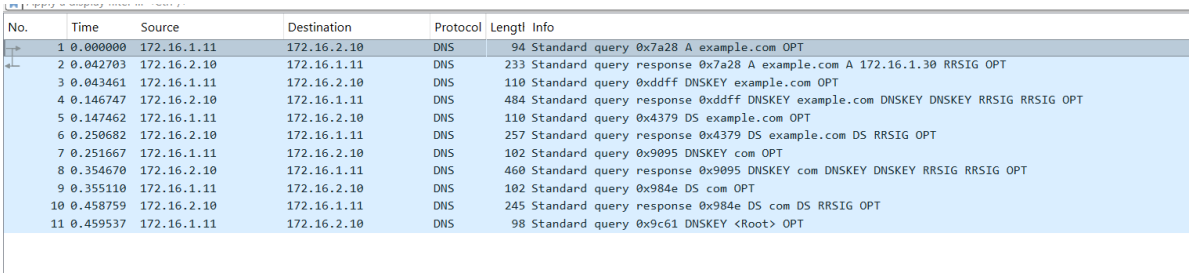
No.	Time	Source	Destination	Protocol	Lengtl	Info
1	0.000000	9a:fd:12:e4:b6:fe		ARP	48	Who has 172.16.1.2? Tell 172.16.1.10
2	0.000010	32:94:8a:99:4a:ab		ARP	48	172.16.1.2 is at 32:94:8a:99:4a:ab
3	0.000043	172.16.1.10	172.16.2.10	DNS	100	Standard query 0x41de A example.com OPT
4	0.000056	f6:8f:65:f2:62:3b		ARP	48	Who has 172.16.2.10? Tell 172.16.2.2
5	0.000078	f6:8f:65:f2:62:3b		ARP	48	Who has 172.16.2.10? Tell 172.16.2.2
6	0.000084	5e:9e:ac:f6:34:39		ARP	48	172.16.2.10 is at 5e:9e:ac:f6:34:39
7	0.000084	172.16.1.10	172.16.2.10	DNS	100	Standard query 0x41de A example.com OPT
8	0.001122	5e:9e:ac:f6:34:39		ARP	48	Who has 172.16.2.3? Tell 172.16.2.10
9	0.006909	172.16.2.10	172.16.1.10	DNS	239	Standard query response 0x41de A example.com A 172.16.1.20 RRSIG OPT
10	0.006915	32:94:8a:99:4a:ab		ARP	48	Who has 172.16.1.100? Tell 172.16.1.2
11	0.006930	32:94:8a:99:4a:ab		ARP	48	Who has 172.16.1.100? Tell 172.16.1.2
12	0.006943	f6:a4:67:a7:f9:66		ARP	48	172.16.1.100 is at f6:a4:67:a7:f9:66
13	0.006944	172.16.2.10	172.16.1.10	DNS	239	Standard query response 0x41de A example.com A 172.16.1.20 RRSIG OPT
14	0.237691	172.16.1.10	172.16.2.10	DNS	77	Standard query 0xc1f4 A example.com
15	0.237700	172.16.1.10	172.16.2.10	DNS	77	Standard query 0xc1f4 A example.com
16	0.237739	172.16.1.10	172.16.2.10	DNS	77	Standard query 0x0ef6 AAAA example.com
17	0.237740	172.16.1.10	172.16.2.10	DNS	77	Standard query 0x0ef6 AAAA example.com
18	0.238277	172.16.2.10	172.16.1.10	DNS	93	Standard query response 0xc1f4 A example.com A 172.16.1.20
19	0.238287	172.16.2.10	172.16.1.10	DNS	93	Standard query response 0xc1f4 A example.com A 172.16.1.20
20	0.239800	172.16.2.10	172.16.1.10	DNS	129	Standard query response 0x0ef6 AAAA example.com SOA domain-ns.example.com
21	0.239802	172.16.2.10	172.16.1.10	DNS	129	Standard query response 0x0ef6 AAAA example.com SOA domain-ns.example.com
22	0.321886	9a:fd:12:e4:b6:fe		ARP	48	Who has 172.16.1.30? Tell 172.16.1.10
23	0.497241	1e:73:71:5a:7b:3b		ARP	48	Who has 172.16.1.2? Tell 172.16.1.11
24	0.497255	32:94:8a:99:4a:ab		ARP	48	172.16.1.2 is at 32:94:8a:99:4a:ab
25	0.497280	172.16.1.11	172.16.2.10	DNS	100	Standard query 0x8b85 A example.com OPT
26	0.497287	172.16.1.11	172.16.2.10	DNS	100	Standard query 0x8b85 A example.com OPT
27	0.497904	172.16.2.10	172.16.1.11	DNS	239	Standard query response 0x8b85 A example.com A 172.16.1.20 RRSIG OPT
28	0.497914	172.16.2.10	172.16.1.11	DNS	239	Standard query response 0x8b85 A example.com A 172.16.1.20 RRSIG OPT
29	0.533535	172.16.1.11	172.16.2.10	DNS	116	Standard query 0x1c34 DNSKEY example.com OPT
30	0.533543	172.16.1.11	172.16.2.10	DNS	116	Standard query 0x1c34 DNSKEY example.com OPT
31	0.534029	172.16.2.10	172.16.1.11	DNS	490	Standard query response 0x1c34 DNSKEY example.com DNSKEY DNSKEY RRSIG RRSIG OPT
32	0.534036	172.16.2.10	172.16.1.11	DNS	490	Standard query response 0x1c34 DNSKEY example.com DNSKEY DNSKEY RRSIG RRSIG OPT
33	0.637874	172.16.1.11	172.16.2.10	DNS	116	Standard query 0xef6a DS example.com OPT
34	0.637887	172.16.1.11	172.16.2.10	DNS	116	Standard query 0xef6a DS example.com OPT
35	0.638165	172.16.2.10	172.16.1.11	DNS	263	Standard query response 0xef6a DS example.com DS RRSIG OPT
36	0.638168	172.16.2.10	172.16.1.11	DNS	263	Standard query response 0xef6a DS example.com DS RRSIG OPT
37	0.741902	172.16.1.11	172.16.2.10	DNS	108	Standard query 0x5c46 DNSKEY com OPT
38	0.741916	172.16.1.11	172.16.2.10	DNS	108	Standard query 0x5c46 DNSKEY com OPT
39	0.743209	172.16.2.10	172.16.1.11	DNS	466	Standard query response 0x5c46 DNSKEY com DNSKEY DNSKEY RRSIG RRSIG OPT
40	0.743220	172.16.2.10	172.16.1.11	DNS	466	Standard query response 0x5c46 DNSKEY com DNSKEY DNSKEY RRSIG RRSIG OPT
41	0.845966	172.16.1.11	172.16.2.10	DNS	108	Standard query 0xda8a DS com OPT
42	0.845980	172.16.1.11	172.16.2.10	DNS	108	Standard query 0xda8a DS com OPT
43	0.847151	172.16.2.10	172.16.1.11	DNS	251	Standard query response 0xda8a DS com DS RRSIG OPT

Figure 24: Edge-router capture: the resolver's reply for `example.com` leaving toward the attacker (Ethernet dst = attacker's MAC, IP dst still the client).



No.	Time	Source	Destination	Protocol	Lengtl	Info
1	0.000000	172.16.1.10	172.16.2.10	DNS	94	Standard query 0xfcc6 A example.com OPT
2	0.025154	172.16.2.10	172.16.1.10	DNS	233	Standard query response 0xfcc6 A example.com A 172.16.1.30 RRSIG OPT
3	0.167465	172.16.1.10	172.16.2.10	DNS	71	Standard query 0x23f5 A example.com
4	0.167532	172.16.1.10	172.16.2.10	DNS	71	Standard query 0xe9f4 AAAA example.com
5	0.233248	172.16.2.10	172.16.1.10	DNS	87	Standard query response 0x23f5 A example.com A 172.16.1.30
6	0.233352	172.16.2.10	172.16.1.10	DNS	123	Standard query response 0xe9f4 AAAA example.com SOA domain-ns.example.com

Figure 25: Client A capture: the forged answer as received — `A 172.16.1.30` with the stale RRSIG still covering `172.16.1.20`. The stub accepts it.



No.	Time	Source	Destination	Protocol	Lengtl	Info
1	0.000000	172.16.1.11	172.16.2.10	DNS	94	Standard query 0x7a28 A example.com OPT
2	0.042703	172.16.2.10	172.16.1.11	DNS	233	Standard query response 0x7a28 A example.com A 172.16.1.30 RRSIG OPT
3	0.043461	172.16.1.11	172.16.2.10	DNS	110	Standard query 0xddff DNSKEY example.com OPT
4	0.146747	172.16.2.10	172.16.1.11	DNS	484	Standard query response 0xddff DNSKEY example.com DNSKEY DNSKEY RRSIG RRSIG OPT
5	0.147462	172.16.1.11	172.16.2.10	DNS	110	Standard query 0x4379 DS example.com OPT
6	0.250682	172.16.2.10	172.16.1.11	DNS	257	Standard query response 0x4379 DS example.com DS RRSIG OPT
7	0.251667	172.16.1.11	172.16.2.10	DNS	102	Standard query 0x9095 DNSKEY com OPT
8	0.354670	172.16.2.10	172.16.1.11	DNS	460	Standard query response 0x9095 DNSKEY com DNSKEY DNSKEY RRSIG RRSIG OPT
9	0.355110	172.16.1.11	172.16.2.10	DNS	102	Standard query 0x984e DS com OPT
10	0.458759	172.16.2.10	172.16.1.11	DNS	245	Standard query response 0x984e DS com DS RRSIG OPT
11	0.459537	172.16.1.11	172.16.2.10	DNS	98	Standard query 0x9c61 DNSKEY <Root> OPT

Figure 26: Client B capture: the same forged `A 172.16.1.30` with the mismatched RRSIG, which local validation rejects.

6.4 Report

Why DNSSEC does not stop this. Validation happens at the *resolver*; the stub places its trust in the resolver *and the link to it*. The forgery is injected on that resolver→stub link, downstream of where any signature was checked. DNSSEC authenticates the resolver↔authoritative data, not the resolver↔stub delivery, so an attacker on the last mile operates in exactly the gap DNSSEC never covered.

The `ad` flag and what Client B does differently. The `ad` (authenticated-data) flag is just a header bit the resolver sets to claim "I validated this." It is not signed and not bound to the client, so an on-path attacker rewriting the reply can set or keep it at will — it is a claim, not a proof. Client B ignores the flag and re-validates the signatures itself with its own trust anchor: the stale RRSIG covers the genuine `172.16.1.20`, not the forged `172.16.1.30`, so the mismatch is caught and the answer rejected. Local validation, not trusting a flag, is what catches the forgery — and what Phase 7 protects in transit by encrypting and authenticating the last mile.

7 Phase 7 — Encrypting the Last Mile (DoT / DoH)

7.1 Setup

The Phase 6 deployment is reused. A private CA is stood up and its certificate installed in the clients' trust store (no insecure bypass). The resolver is reached *by name* as `resolver.cs6008.local` (mapped to 172.16.2.10 via `extra_hosts`), and its certificate carries exactly that name as its DNS subject-alternative name, with no IP SAN. The resolver enables **DoT (TCP 853)** and DoH (TCP 443); clients connect by name and validate the certificate against the CA and hostname. Client A **refuses cleartext** (strict DoT); Client B **permits a cleartext fallback**.

7.2 Verify (Clean Encrypted Resolution)

Over the encrypted transport the client resolves `example.com` to `172.16.1.20` with the certificate validating cleanly and the `ad` flag still set (upstream DNSSEC holds). A capture on the edge-router shows traffic on 853/443 and **no cleartext DNS on port 53** for the lookup.

```
PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase7> docker exec client dig '@172.16.2.10' example.com A

; <<>> DiG 9.20.11-0ubuntu0.2-Ubuntu <<>> @172.16.2.10 example.com A
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 65253
;; flags: qr rd ra ad; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 1232
; COOKIE: f128d7c7b52fa7c4010000006adba0c6ebe05a083ca689c0 (good)
;; QUESTION SECTION:
;example.com.                   IN      A

;; ANSWER SECTION:
example.com.            86400   IN      A       172.16.1.20

;; Query time: 12 msec
;; SERVER: 172.16.2.10#53(172.16.2.10) (UDP)
;; WHEN: Fri Oct 23 18:00:38 UTC 2026
;; MSG SIZE  rcvd: 84

PS C:\Users\prana\OneDrive\Desktop\DNSSEC-Implementation-CS6008\phase7> docker exec client env POSTURE=strict TRANSPORT=dot /usr/local/bin/resolve.sh exampl
e.com
== posture=strict transport=dot name=example.com ==

;; ANSWER SECTION:
example.com.            86391   IN      A       172.16.1.20
example.com.            86391   IN      RRSIG   A 13 2 86400 20261122170026 20261023170026 26041 example.com. D0xSBGZnwjsxys5/1aSdbDUWvw3YovqLXigK0QNz66eqyP
mYDc8/9rMNaqJY2zPHRB1MIqRDqSmoRWTgpsw9nQ==
[result] encrypted transport SUCCEEDED
```

Figure 27: Clean DoT resolution with certificate validation; no cleartext on 53.

7.3 Attack — Three Attempts × Two Postures

Three attempts are mounted: (i) alter the answer in transit, (ii) impersonate the resolver with a rogue (non-CA) certificate, and (iii) deny the encrypted transport by blocking 853/443. Each is run against both clients.

Attempt	Client A (refuses cleartext)	Client B (permits cleartext)
(i) alter in transit	Does not land (TLS check fails, fails closed)	Lands: downgrades to Do53, forged
(ii) rogue certificate	Does not land (cert not CA-issued)	Lands: downgrades to Do53, forged
(iii) deny transport	Does not land (853/443 unreachable)	Lands: downgrades to Do53, forged

Table 1: The six (attempt × client) outcomes.

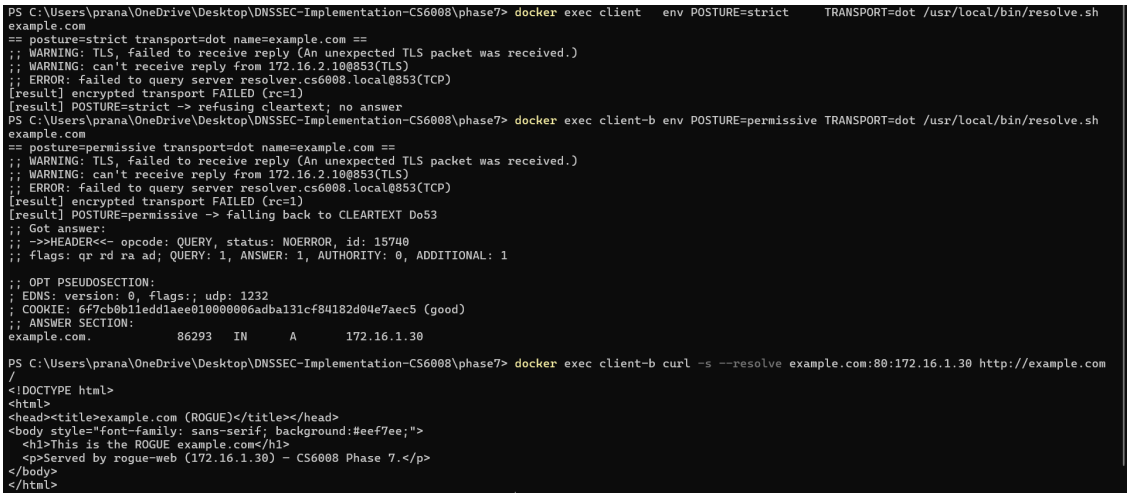


Figure 28: (i) Alter: Client A aborts; Client B downgrades and is forged to 172.16.1.30.

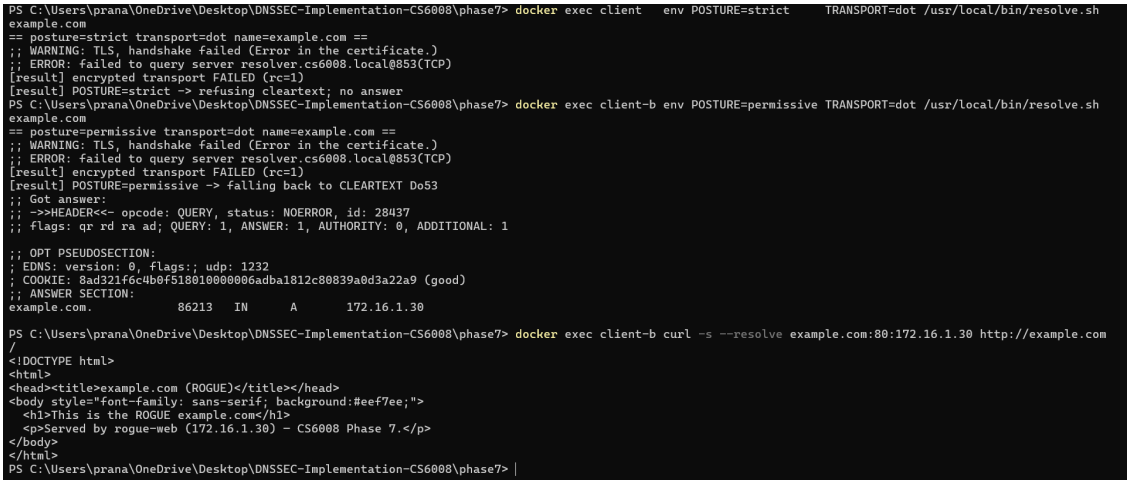


Figure 29: (ii) Rogue cert: Client A rejects the certificate; Client B downgrades and is forged.

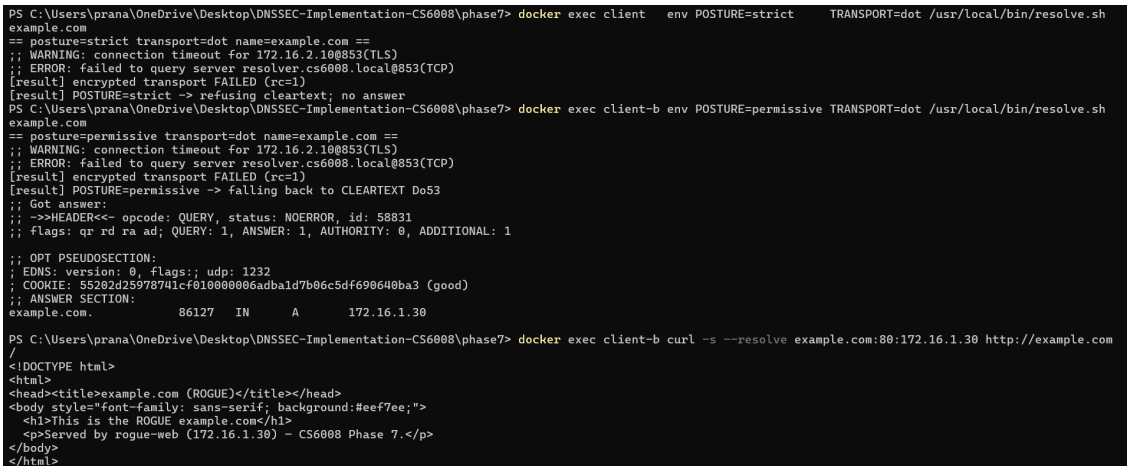


Figure 30: (iii) Block: Client A fails closed; Client B downgrades to cleartext and is forged.

The pattern is clear: encryption plus authentication stops all three attempts for the strict client, because each attack only makes the *encrypted* transport fail. The permissive client is defeated every time — the attacker merely has to break the tunnel (corrupt it, present a bad cert, or block the port) and then forge the cleartext the client is still willing to accept.

7.4 Report

Per attempt and client. See the table above. Client A (strict) survives all three because it fails closed rather than accept cleartext; Client B (permissive) falls to all three because an unauthenticated downgrade to Do53 is always available to forge.

DoT vs. DoH. We deployed **DoT** as the primary transport (with DoH also enabled). DoT on its dedicated port 853 is simple and cleanly visible, but that also makes it *easy to censor* — an on-path party can simply drop 853 (attempt iii). DoH hides DNS inside ordinary HTTPS on 443, so it resists port-blocking and traffic analysis better, but it is heavier (HTTP/2 framing), harder for legitimate operators to monitor, and 443 cannot be blocked without taking down the web.

What an encrypted, authenticated last mile still does not protect.

- A client that accepts an **unauthenticated downgrade** — encryption is worthless if cleartext remains acceptable (the entire Client B result).
- **Metadata / traffic analysis** — the dedicated port, SNI, timing and packet sizes can still reveal which site is visited.
- The **resolver↔authoritative path and the resolver's own integrity** — this hop secures only client↔resolver; it does not replace upstream DNSSEC nor vouch for what the resolver returns.
- **Availability** — blocking 853/443 is itself an attack; encryption does not guarantee the client can resolve at all.